

5. 개발자를 위한 프롬프트엔지니어링



1) AI 개발 도구의 이해



AI 개발 도구란

- AI 개발 도구는 인공지능 기술을 활용해 소프트웨어 코딩, 디버깅, 테스트, 문서화 등 개발 전 과정의 반복 작업을 자동화하고 생산성을 높여주는 소프트웨어입니다.
- 대표적으로 코드 자동완성, 자연어 기반 코드 생성, 오류 수정 기능 등을 지원합니다.
- 주요 기능
 - 코드 생성 및 자동완성: 자연어 프롬프트나 주석을 바탕으로 필요한 코드 스니펫이나 함수 전체를 작성합니다.
 - 디버깅 및 리팩토링: 코드의 오류를 찾아 수정 방안을 제안하고, 코드를 더 효율적으로 정리합니다.
 - 자동 테스트 및 문서화: 테스트 케이스를 자동으로 생성하거나 코드에 대한 자연어 설명을 붙여 문서를 만듭니다.

작업 방식에 따른 AI 개발 도구 유형

구분	IDE 확장형 (Extension)	AI 네이티브 IDE	터미널/CLI 기반 도구
개념	기존 에디터(VS Code 등)에 플러그인 형태로 설치	개발 환경 설계 단계부터 AI 상호작용을 중심에 둔 통합 툴	터미널 창에서 명령줄 인터페이스로 직접 소통
작업 방식	코드 에디터 옆 패널에서 챗(Chat) 또는 인라인 자동완성	프로젝트 전체 인덱싱, 멀티 파일 동시 수정, 에이전트 모드 지원	파일 생성, 터미널 명령 실행, Git 관리까지 자율 수행
장점	기존 개발 환경 및 세팅을 그대로 유지 가능	직관적인 UI/UX, 가장 편리한 파일 단위 제어	빠르고 가벼움, 자동화 스크립트 및 에이전틱 작업에 유리
대표 예시	GitHub Copilot, Continue, Cline	Cursor, Windsurf Google Antigravity	Claude Code, Aider, OpenHands

AI 개발 도구 최신 현황 및 트렌드

- AI 코딩 도구의 진화 흐름: '코드 제안'에서 '자율 에이전트'로
 - 1세대: 단순 자동완성 (Auto-completion)
 - 개발자가 치는 코드의 다음 몇 줄을 예측하여 추천 (단일 라인 중심)
 - 2세대: 대화형 어시스턴트 (Chat & Edit)
 - 사이드 패널에서 질의응답을 하고, 특정 파일의 코드를 복사·붙여넣기 하거나 패치 적용
 - 3세대: 에이전틱 코딩 (Agentic Workflow - 현재)
 - 멀티 파일 수정: 지시 하나로 프론트엔드, 백엔드, 설정 파일을 한 번에 연계 수정
 - 자가 디버깅(Self-healing): 터미널 명령을 직접 실행하고 에러 발생 시 스스로 코드를 고쳐 재시도
 - 플랜 모드(Plan Mode): 복잡한 작업 시 바로 코딩하지 않고 계획서를 먼저 수립한 뒤 단계별 승인을 받으며 구현

2) PRD 기반의 프롬프트 작성



PRD(제품 요구사항 정의서)의 개념

■ PRD(Product Requirements Document)

- 제품을 만들거나 업데이트하기 위해 기능을 기획하는 단계에서 요구사항을 개괄적으로 설명하는 문서
- 프로젝트 매니저 또는 개발자들이 코딩에 앞서 작성하는 제품 개발 프로세스 전반에 걸쳐 필수적인 중요 문서

■ 단계별 PRD의 활용 방식

- 기획 단계 : 제품의 비전과 전략을 정의
- 디자인 단계 : 제품이 사용자 요구 사항을 충족하는지 확인
- 개발 단계 : 기능의 목적과 구현을 확인
- 테스트 단계 : 제품이 기술 요구 사항을 충족하는지 확인
- 출시 단계 : 제품이 시장 혹은 사용자의 의 요구 사항을 충족하는지 확인

- PRD를 작성하면 개발팀 및 이해관계자에게 공유하고 제품 개발 프로세스에 참여하는 모든 사람이 제품의 요구 사항을 확인하고 개발에 착수할 수 있도록 합니다.

PRD는 좋은 프롬프트

PRD = 제품 기획서

개발자들이 코딩 전에 작성하는 문서

PRD

무엇을 만들지 명확히 정의

프로젝트 목적과 배경 제시

요구사항, 구현 방법 구체화

=

좋은 프롬프트

해야 할 일을 구체적으로 제시

왜 필요한지 어떤 상황인지 설명

필요한 기능과 조건 세부 명시

PRD 작성법(1)

- 먼저 명확한 목표를 설정해야 합니다.

- 무엇을 만들지 한 문장으로 정의하고 성공을 측정할 수 있는 구체적인 지표를 함께 제시 합니다.
- 예: 회원가입/로그인 기능을 만든다면 이메일 인증 완료율이나 비밀번호 복구 성공률 같은 수치 목록을 명시 합니다.

목표

사용자가 이메일과 비밀번호로 회원가입하고 로그인 할 수 있다.

성공 기준

- 회원 가입 후 이메일 인증 완료율 80% 이상
- 로그인 실패 시 명확한 오류 메시지 제공
- 비밀번호 찾기 기능으로 24시간 내 90% 복구

PRD 작성법(2)

■ 구체적인 요구 사항을 작성합니다.

- 각 기능에 고유한 식별자를 부여하고, 입력 필드는 무엇인지, 어떤 검증 규칙을 적용할 지, 예외상황은 어떻게 처리할지 등을 상세히 기술합니다.
- 회원가입이라면 이메일 형식 검증, 비밀번호 규칙, 중복체크 같은 세부 사항을 모두 담아야 합니다.

기능 요구사항

FR-001: 회원 가입

- 입력 필드: 이메일, 비밀번호, 비밀번호 확인, 닉네임
- 이메일 형식 검증(정규식)
- 비밀번호 규칙: 8자 이상, 영문+숫자+특수문자 조합
- 중복 이메일 체크
- 가입 완료 시 인증 이메일 발송

PRD 작성법(3)

- 기술 명세서에는 실제 구현에 필요한 기술적 결정 사항을 명시합니다.
 - 인증 방식은 무엇을 쓸지, 토큰 만료 시간은 어떻게 설정할지, API 엔드포인트는 어떤 구조로 만들지 등을 표로 정리하면 개발자들이 한 눈에 파악하기 쉽습니다.

기능 요구사항

인증방식

- JWT(Access Token + Refresh Token)
- Access Token: 15분 만료
- Refresh Token: 7일 만료

API 엔드포인트

메서드	경로	설명
POST	/auth/register	회원가입
POST	/auth/login	로그인
POST	/auth/logout	로그아웃

PRD 작성법(4)

■ 제약사항과 범위를 명확히 구분해야 합니다.

- 이번 단계에서 포함할 기능과 제외할 기능을 분명히 나누어 작성하면, 범위 확대를 방지하고 팀의 집중력을 유지할 수 있습니다.
- 예를 들어 소셜 로그인이나 2단계 인증 같은 기능은 다음 단계로 미루는 것도 좋은 전략입니다.

범위

포함(In Scope)

- 이메일/비밀번호 인증
- 비밀번호 찾기/재설정
- 기본 프로필(닉네임)

제외(Out of Scope)

- 소셜로그인
- 2FA
- 프로필 이미지

5W1H 프레임워크

- 다음 질문들에 답하다 보면 막연한 아이디어가 구체적인 기획으로 발전합니다.

Why

왜 만드는가?
목적과 동기

Who

누구를 위한 것인가?
대상 사용자

What

무엇을 만들 것인가?
핵심 기능과 내용

When

언제 사용되는가?
사용시점과 상황

Where

어디서 사용되는가?
사용환경과 플랫폼

How

어떻게 만들 것인가?
구현 방식과 기술

5W1H기반의 PRD

- 좋은 제품 기획서를 만들기 위해서는 체계적인 사고가 필수입니다.
- 5W1H 원칙을 PRD(제품 요구사항 문서)에 적용하면 기획의 의도와 실행 방안을 누구나 직관적으로 이해할 수 있게 구조화할 수 있습니다.
 - Why로 목적을 명확히 하고, Who로 대상을 정의하며, What으로 기능을 구체화 합니다.
 - When과 Where로 사용 맥락을 파악하고, How로 구현방법을 결정합니다.
 - 프로그램, 웹 페이지, 애플리케이션 등의 소프트웨어는 기능과 요구사항을 명세화하는 PRD 구조를 따르는 것이 적합합니다.

5W1H기반의 PRD 항목별 포함 내용

Why (목적 및 배경)	Who (타겟 사용자)	What (기능 요구사항)
- 질문: 이 제품(기능)을 왜 만드는가? - 내용: 해결하려는 사용자의 페인 포인트(Pain Point), 시장의 기회, 비즈니스 가치. - 핵심: '정당성' 부여. 이 작업이 우선순위가 높은 이유를 설명합니다.	- 질문: 누가 이 기능을 사용하는가? - 내용: 주요 페르소나(Persona), 사용자 역할 (관리자, 일반 유저 등), 이해관계자. - 핵심: 사용자의 특성을 정의하여 UI/UX 방향성을 결정합니다.	- 질문: 무엇을 개발하는가? - 내용: 핵심 기능 목록, 사용자 스토리, 성공 지표(KPI). - 핵심: 구현해야 할 결과물의 실체를 명확히 합니다.
When (일정 및 마일스톤)	Where (범위 및 플랫폼)	How (구현 방식 및 상세 로직)
- 질문: 언제 출시하며, 주요 단계는 언제인가? - 내용: 개발 시작일, 테스트 기간, 최종 배포일, 단계별 마일스톤. - 핵심: 프로젝트의 타임라인을 공유하여 리소스 관리를 가능하게 합니다.	- 질문: 어느 환경에서 작동하며, 어디까지가 범위인가? - 내용: 지원 OS(iOS, Android), 웹/모바일 여부, 서비스 내 노출 위치. - 핵심: 개발 범위를 한정하여 불필요한 리소스 낭비를 방지합니다.	질문: 어떻게 작동하며, 어떻게 구현할 것인가? 내용: 상세 정책, 예외 케이스 처리, 데이터 플로우, 디자인 와이어프레임. 핵심: 개발자와 디자이너가 실제 작업에 착수할 수 있는 상세 가이드를 제공합니다.

기본 질문 템플릿

- 5W1H를 활용하여 범용적인 기본 질문 템플릿 만들기
 - 5W1H의 모든 요소가 적절히 포함되어야 합니다.
 - 구현 방향을 구체적으로 요구해야 합니다.
 - AI를 위해 구체적인 기능을 기술적으로 설명해야 합니다.
- 효과적인 프롬프트를 위한 기본 질문 템플릿

[무엇]을 만들려고 해.

주요 기능은 [구체적인 기능 설명] 이야.

[누구]를 위한 것이고,

[왜/어떤 문제를 해결]하기 위해 필요해.

어떻게 구현하면 좋을지 기술적인 방향과 함께 구체적인 PRD를 작성해 줘

5W1H 기반의 프롬프트 작성 예시

포트폴리오 웹사이트 PRD (Product Requirements Document)

1. 제품 개요

- **목적:** 30초 내에 지원자의 기술적 핵심 역량과 문제 해결 능력을 증명하여
- **타겟:** 기술 면접관 및 채용 담당자.
- **핵심 가치:** 가독성, 기술적 깊이, 사용자 경험(UX).

2. 주요 기능 (MVP)

- **히어로 섹션 (The 5-Second Rule):** 한 문장으로 정의된 핵심 기술 스택과 강점 (예: "성능 최적화에 집요한 풀스택 개발자").
- **핵심 프로젝트 카드:** 단순 나열이 아닌 '도전 - 해결 - 성과' 위주의 3개 이내 프로젝트.
- **기술 스택 시각화:** 단순 아이콘 나열보다 숙련도를 보여주는 방식 채택.
- **라이브 데모 및 코드 링크:** 즉시 확인 가능한 배포 링크와 정돈된 GitHub Repository.

(P)너는 10년차 경력의 웹사이트 기획자야.

(C)나는 취업준비를 위해 포트폴리오 웹 사이트를 만들려고 해.

(T)주요 기능은 채용 담당자가 30초 안에 내 역량을 파악할 수 있는 임팩트 있는 구성이야. 풀스택 개발자로서의 신입사원 채용을 준비하는 것이고, 경쟁력 있는 지원자로 보이기 위한 내용이 필요해.

(F)어떻게 구현하면 좋을지 기술적인 방향과 함께 PRD를 작성해 줘.

역할지정

- 역할 지정 시 역할의 관점으로 응답하므로 답변의 깊이, 목적, 대상 등 여러 가지 정보를 한 번에 주입하는 효과가 있습니다.
- 자주 사용되는 페르소나

페르소나	특징	작업
시니어 개발자	깊은 기술 지식, 예외 상황 인식	코드 리뷰, 아키텍처 설계
주니어 개발자	기본에 충실, 명확한 설명	문서화, 튜토리얼 작성
QA엔지니어	버그 찾기, 테스트케이스	테스트 작성, 디버깅
DevOps 엔지니어	배포, 인프라, 자동화	CI/CD 설정, 도커와 쿠버네티스 구성
보안 전문가	취약점 분석, 보안 강화	보안 감사, 인증 구현
테크니컬 라이터	명확한 문서화	README, API 문서

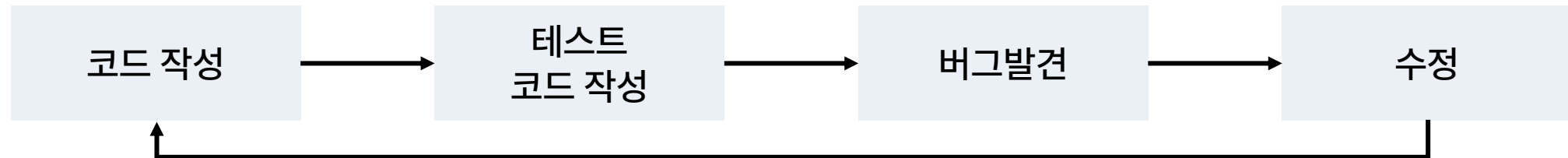
3) 테스트 주도 개발(TDD)



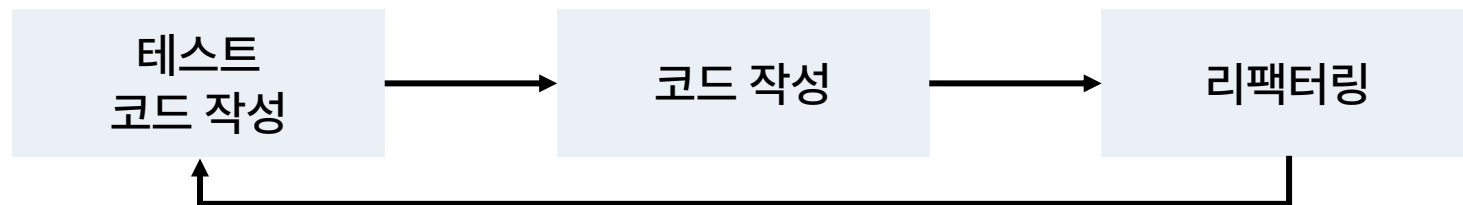
TDD(Test-Driven Development, 테스트 주도 개발)

- 실제 코드를 작성하기 전에 실패하는 테스트 코드를 먼저 작성하고, 그 테스트를 통과시키는 최소한의 코드를 만든 뒤 리팩토링하는 과정을 반복하는 소프트웨어 개발 방법론입니다.

- 전통적인 개발 순서



- 테스트 주도 개발



TDD 개발 주기 (Red-Green-Refactor Cycle)

- 테스트 코드를 먼저 작성하고 이를 바탕으로 실제 코드를 점진적으로 완성해나가는 TDD의 표준 작업 절차입니다.
 - Red (실패): 구현하고자 하는 기능의 기대 동작을 정의하는 실패하는 테스트 코드를 먼저 작성합니다.
 - Green (성공): 작성한 테스트를 통과할 수 있을 만큼의 최소한의 실제 코드를 작성합니다.
 - Refactor (리팩토링): 중복 코드를 제거하고 코드를 깔끔하고 효율적으로 개선합니다

TDD의 장점과 단점

■ 장점:

- 코드의 버그를 미리 줄이고 안정성을 높입니다.
- 개발 중 방향성을 잃지 않게 돕고, 견고한 설계 및 문서화 효과를 제공합니다.

■ 단점:

- 초기 개발 속도가 느려질 수 있습니다.
- 테스트 코드 작성에 대한 추가적인 학습과 비용이 필요합니다.

바이브코딩(Vibe Coding)과 TDD

- 세부 구현을 AI에게 맡기되, AI가 지켜야 할 명확한 스펙과 가드레일(테스트 코드)을 먼저 설정하는 개발 방식입니다.
- 자연어로 빠르게 결과물을 만드는 바이브 코딩 특성상 발생하기 쉬운 환각(Hallucination)과 의도치 않은 버그를 방지하는 핵심 안전장치 역할을 합니다.

바이트 코딩에서 TDD의 주요 진행 흐름

1) 테스트 요구사항 프롬프팅 (Red)

- AI에게 기능 구현 전에 "이러한 입력과 조건에서 기대하는 결과"에 대한 테스트 코드를 먼저 작성하도록 요청합니다.

2) 기능 구현 및 테스트 통과 (Green)

- AI에게 해당 테스트를 모두 통과시키는 최소한의 구현 코드를 생성하도록 지시합니다.

3) 리팩토링 및 반복 (Refactor)

- 테스트가 통과되면 AI에게 코드 품질 개선, 최적화 등을 지시하며 기존 테스트가 깨지지 않는지 즉시 검증합니다.

바이트 코딩에서 TDD가 주는 이점

- 프롬프트 명확화: 모호한 설명 대신 구체적인 입·출력 예시가 테스트 코드로 정리되어 AI의 이해도가 높아집니다.
- 빠른 디버깅: 기능이 오작동할 때 장황한 대화 없이 실패한 테스트 로그만 AI에 전달해 즉시 수정할 수 있습니다.
- 코드 신뢰성 확보: 개발자가 세부 코드를 일일이 검토하지 않아도 기능의 정상 작동 여부를 빠르게 보장받습니다.

TDD 방식의 예시

- AI에게 테스트 코드를 작성해달라고 하는 것만으로도 바이트 코딩의 질을 향상 시킬 수 있습니다.
 - 다음은 '로그인 기능'을 만들기 위해 단계별로 AI에게 입력하는 프롬프트입니다.

사용자 로그인 기능을 TDD 방식으로 구현합니다.

TDD 단계

1. 먼저 테스트 케이스를 작성해 주세요.
2. 테스트가 실패하는 것을 확인한 후 구현 코드를 작성해 주세요.
3. 테스트가 통과하면 코드를 리팩터링해 주세요.

테스트케이스 요구사항

- 올바른 이메일/비밀번호로 로그인 성공
- 잘못된 이메일로 로그인 실패
- 잘못된 비밀번호로 로그인 실패
- 빈 이메일 입력 시 오류
- 빈 비밀번호 입력 시 오류



THANK YOU!